------------------------- Slip 5 --------------------

Q.1) Read the current directory and display the name of the files, no of files in current directory

```c
#include <stdio.h>
#include <dirent.h>

int main() {
    DIR *dir;
    struct dirent *entry;
    int count = 0;

    // Open current directory
    dir = opendir(".");
    if (dir == NULL) {
        perror("opendir");
        return 1;
    }

    printf("Files in the current directory:\n");

    while ((entry = readdir(dir)) != NULL) {
        // Ignore "." and ".."
        if (entry->d_name[0] != '.') {
            printf("%s\n", entry->d_name);
            count++;
        }
    }

    closedir(dir);

    printf("\nTotal number of files: %d\n", count);

    return 0;
}
```

Q.2) Write a C program to create an unnamed pipe. The child process will write following three messages to pipe and parent process display it.

Message1 = “Hello World”

Message2 = “Hello SPPU”

Message3 = “Linux is Funny”

```c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>

int main() {
    int fd[2]; // fd[0] = read end, fd[1] = write end
    pid_t pid;
    char buffer[100];

    // Create unnamed pipe
    if (pipe(fd) == -1) {
        perror("pipe");
        exit(1);
    }

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        // -------------------------
        // Child Process: write to pipe
        // -------------------------
        close(fd[0]); // Close read end
```

```
        char *messages[] = {
            "Hello World",
            "Hello SPPU",
            "Linux is Funny"
        };

        for (int i = 0; i < 3; i++) {
            write(fd[1], messages[i], strlen(messages[i]) + 1); // +1 for null terminator
        }

        close(fd[1]); // Close write end
        exit(0);
    } else {
        // -------------------------
        // Parent Process: read from pipe
        // -------------------------
        close(fd[1]); // Close write end

        printf("Parent reading messages from pipe:\n");

        for (int i = 0; i < 3; i++) {
            read(fd[0], buffer, sizeof(buffer));
            printf("%s\n", buffer);
        }

        close(fd[0]); // Close read end
    }

    return 0;
}
```